

AI Conversational Chatbot - Assistant

By

Ilnara Temerbulatova

A Final Report

Submitted in Partial Fulfilment of the Requirements for the Degree of

Bachelor's degree

in

Computer Science and Electrical Engineering

University of Essex

Colchester, April 2026

ABSTRACT

This project presents the design and implementation of **Aura**, an AI-powered conversational assistant accessible through both a web application and a Telegram chatbot. The system integrates modern artificial intelligence capabilities using Google's Gemini API to provide intelligent responses to user queries.

The main goal of the project was to build a conversational assistant that works on two platforms and capable of interacting with users in natural language while providing realtime responses such as weather and news. The system architecture combines a Flaskbased backend server, a JavaScript fronted, and a Telegram bot that communicates with the server via HTTP requests.

On the features side, system includes conversation history, message saving, AI-powered responses, external API integrations for weather and news data, and Google login. The project also explores user interface design considerations for conversational systems and the integration of AI services into real-time applications.

The final system demonstrates how modern AI technologies can be integrated into practical software systems that operate across multiple platforms. The project highlights both the technical challenges and design considerations involved in building a scalable AI chatbot application.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my project supervisor for their guidance and valuable feedback throughout the development of this project.

I would also like to thank the University of Essex for providing the resources and academic environment that made this project possible.

Finally, I would like to acknowledge the support of my family and friends for their encouragement and motivation during the development and completion of this proje

TABLE OF CONTENT

	Page
Chapter 1. Introduction	5
Chapter 2. Background and literature review	7
Chapter 3. System Design and Architecture	10
Chapter 4. Implementation	13
Chapter 5. System Features and Functionality	22
Chapter 6. Testing and Evaluation	27
Chapter 7. Project Planning and Risk Management	32
Chapter 8. Project Context	37
Chapter 9. Results and Discussion	39
Chapter 10. Conclusion	42
REFERENCE LIST	44

Chapter 1. INTRODUCTION

Over the past few years, Artificial intelligence has become an integral part of digital systems, particularly through the development of conversational interfaces known as chatbots. These systems let people talk to software in their own words, which makes apps easier to use, more accessible, and faster. A big part of that shift has come from large language models like ChatGPT and Google Gemini, which are good enough to hold a proper conversation and handle a wide range of tasks [2]. This made me want to explore what it actually takes to build something similar from scratch.

The main idea of this project is to design and implement an AI-powered conversational assistant named **Aura**. The system is developed to work on two-sided platforms like Web interface and Telegram chatbot. The project shows how intelligent assistants can be added to real-world apps to make them easier to use and automatically get information by combining AI capabilities with modern web technologies [1].

The motivation for choosing this project is to meet the growing need for smart digital assistants that can give real-time information and help on many different platforms for better user experience. People who use traditional software often face with complicated menus or commands in a certain way. A Conversational interface removes that barrier; it makes day to day tasks simpler. This makes user experience easier to handle and more accessible to a wider range of people [3].

The **Aura** assistant meets this need by using a number of important technologies, such as the Flask framework for backend development, a web interface based on JavaScript for user interaction, and the Telegram Bot API for chatbot features. The system also uses external APIs to add useful features like getting news and weather information. Also, system connects to Google services, which lets users use the Gmail API and Google Calendar API to access email and calendar features.

The main objectives of this project:

- To develop a conversational AI system that can understand and answer queries in natural language

- To create a solution that works on two platforms and can be accessed through a web interface and a Telegram chatbot
- To add features to the assistant by connecting it to outside services like weather and news APIs
- To implement user authentication using Google login, which lets safely access personalized features
- To integrate Google services like email and calendar, so users can access and manage information directly through the assistant
- To design and develop a user-friendly interface that is easy to use and understand and allows for smooth and natural interaction

The scope of the project is focused to be a working prototype rather than commercial product. The system covers the important features such as natural interaction, real-time data retrieval, and message management, but it does not attempt to compete with all known systems. Instead, the aim of the project was to focus and understand the design and implementation, and to develop something that actually functions.

This report walks through the whole development process of the Aura assistant, from the background research, design decisions to implementation, and testing. Moreover, it discusses the challenges that have been faced during this project process, as well as the ethical and practical issues related to AI-based systems. Since working with user data and AI generated responses raises issues worth thinking about carefully.

Chapter 2. BACKGROUND AND LITERATURE REVIEW

This chapter represents the background and literature review about conversational AI and chatbot technologies. It looks at how conversational AI systems work, how they develop over time, and what existing research says about building effective chatbot applications. This review and research helped to make decisions throughout the development of the AURA assistant.

2.1 Introduction to Conversational AI

Conversational AI system is the technology that allows interaction between users and machines using natural language. These systems are commonly implemented as chatbots, which can help people do a variety of tasks.

Nowadays chatbots have come long way from rule-based to AI intelligent systems. Chatbot systems from the past were not very flexible because they used pre-set rules and scripted responses. However, recent advancements in machine learning and natural language processing have made systems that are more advanced and adaptable [8].

2.2 Chatbot Architectures

The architecture of chatbot systems plays a crucial role that affects how well it works and how easy it is to add new features. Most modern systems adopt a modular structure consisting of user interfaces, processing units, and external data sources.

According to Mohammed and Aref [1], chatbot architectures usually have parts for input processing, response generation, and external service communication. Similarly, Zhou et al. [2] point out the importance of designing conversational systems that can manage context and maintain user engagement.

2.3 LLM in Chatbots

Recent developments in large language models have significantly improved chatbot functionalities. Google Gemini and other models like it allows systems to generate responses that are aware of the situation and sound like a person.

Aldhafeeri explains that generative AI models are commonly used in modern conversational systems due its ability to handle large amounts of data and produce useful results [4]. These models can handle complex queries and work with different range of applications, from information retrieval to automating tasks.

However, LLM systems may also retrieve inaccurate or unclear responses, which shows how important it is to test and validate systems during the process. This is crucial aspect when designing and evaluating AI systems.

2.4 Integration of External Services

Nowadays chatbot systems often use external APIs to extend their functionality. These services allow chatbots to provide real-time information, such as weather, news, and access to personal data.

According to Madi and Al-Ajlan [5][9], the integration of external APIs into chatbot systems significantly extends their functionality, enabling them to perform practical, real-world tasks beyond basic conversation. This approach allows chatbots to act as central platforms for accessing multiple services, it becomes a useful assistant on a daily basis.

In addition, modern systems often use APIs such as the Telegram Bot API [6] and AI services like the Gemini API [7]. These tools support communication and response generation. These technologies play an important role in building systems that can grow and change.

2.5 Chapter Summary

The literature research shows that chatbot systems depend on advanced AI models, modular architectures, and integration with external services. Even though current systems are good at having conversational abilities, there is a growing need for multiplatform assistants that are easy to use, accessible, and useful.

This project fills this gap by developing the AURA assistant, which integrates conversational AI with web and messaging platforms, along with external service integration.

The main goal is to offer user-friendly solution that is in line with the latest trends and technologies.

Chapter 3. SYSTEM DESIGN AND ARCHITECTURE

The Aura assistant is designed using a client-server architecture, which holds the communication between user interfaces and processing system. This approach was chosen in order to it is flexible, scalable, and appropriate for applications that require to handle data in real-time data and interact with external services [1].

3.1 Backend Server Design

The backend server is implemented using the Flask web framework. It was chosen because its lightweight structure and integration with external APIs, making it suitable for developing web-based service [7]. It handles incoming HTTP requests from both platforms web interface and Telegram bot, and it acts as the hub for all API requests going in and out of the system.

This server is responsible for:

- receiving user requests from web interface and Telegram bot
- communication with AI service through Google Gemini
- API integration like weather, news, email and calendar
- Receiving responses to an appropriate request
- Managing conversation history and user data

3.2 Interface Design

Web-based

The web interface is developed by using HTML, CSS and JavaScript in order to provide a better user experience based on chat environment. Due to its simplicity and intuitiveness, it is made to be used in a natural way, which makes a new user to be able to start a conversation immediately, without needing to figure out how anything works. As my project Aura follows these principles to get desired outcomes.

The key features of interface:

- conversational chat-based interface
- side bar navigation
- saved messages
- local storage for chat history

Telegram Bot

This system also includes the Telegram-based chat bot by using appropriate API, so it allows users to communicate through messaging [6]. This bot uses HTTP requests to interact with the backend server; it opens one more option of user interface. The process is conceptually comparable to a web-based interface, but the user exchanges messages over Telegram. This feature increases accessibility by enabling communication outside of the online system.

3.3 External API's

In order to improve the functionality of the Aura assistant, it combines some external APIs. To produce real-time, relevant data, system makes use of a huge language model using the Google Gemini API, since they can generate context-aware responses that hold up across a conversation [4]. On top of that, weather and news APIs let the system fetch live data, and the Gmail and Google Calendar integrations add some personal productivity features.

System integrates following services:

- Receive AI responses using Google Gemini
- Weather APIs for real-time weather information
- News APIs for retrieving current news
- Email access using Gmail API
- Calendar access using Google Calendar API

3.4 Data Flow and System Interaction

This is the structure of process, that system follows:

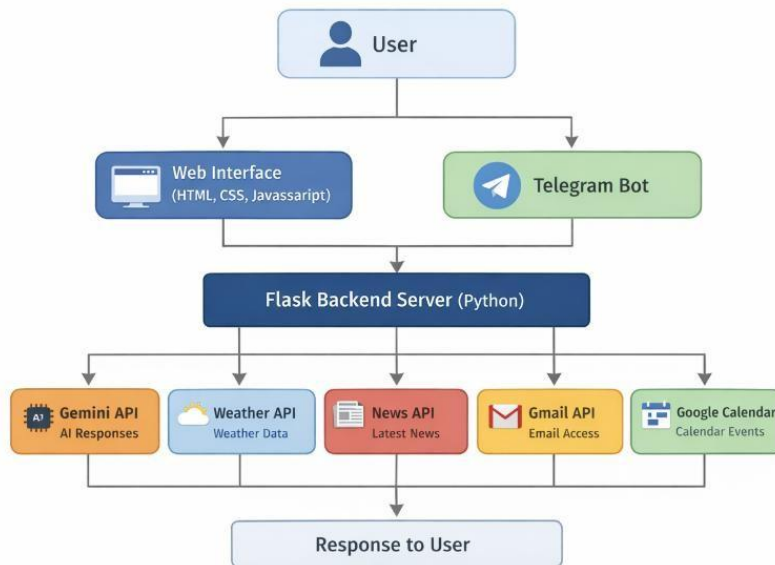


Figure 1. System Architecture of the Chatbot

Figure 1 shows the overall structure of the system. It consists of three main parts: the user interfaces, the backend server, and external APIs. Users could communicate with this system through a web-based interface or Telegram chatbot. And both of this system sends the user queries to the backend server, which evaluates the user input, sends to the request to the external services and then returns the appropriate response. This structure guarantees the handling of user requests and enables the system to send accurate responses in real-time.

3.5 Chapter Summary

Overall, the system architecture shows a scalable and modular approach to creating AI-powered applications, facilitating integration with various platforms and external services while preserving effective communication between system components.

Chapter 4. IMPLEMENTATION

This chapter describes the implementation of the AURA assistant, focusing on the technologies used and how the system parts were developed and integrated. The system follows a modular approach, consisting of a backend server, frontend interface, and external integrations.

4.1 Development Environment and Technologies

The AURA assistant was developed in a combination of variety programming languages, frameworks, and external services. This approach was chosen because of ease of use, flexibility and ability to provide API connection and real-time interaction. It can be seen on table 1 below:

Technology	Purpose	Reason for Selection
Python	Backend development	Simple and readable
Flask	Web framework	Lightweight and suitable for API integration
HTML/CSS	Frontend design	Simple
JavaScript	Client-side interaction	Dynamic and real-time communication
Google Gemini API	Generation of AI responses	Good language processing
Telegram API	Bot messaging interface	One more platform for interaction
Weather/News API	Weather and news retrieval	Access to real-time information

Table 1. Technologies used in AURA assistant

Python was used for backend implementation because of its readability and ability to support web frameworks and AI integration. The main reason of selecting Flask framework is its lightweight structure, it applies best approach when it comes to creating API based apps.

When it comes to the system's frontend, it was developed by using HTML, CSS and JavaScript. Due to its characteristics that they can offer such as, simple and efficient method of developing interactive web interface. When we talk about asynchronous interaction between backend and frontend, the best approach to handle this is using JavaScript.

Generation of AI responses were produced by using the Google Gemini API, that can offer appropriate natural language processing capabilities for conversational systems. In order to provide platform for communication through messages, it was used Telegram Bot API.

Also, some external APIs like weather and news, were integrated in order to improve system functionality to provide real-time data.

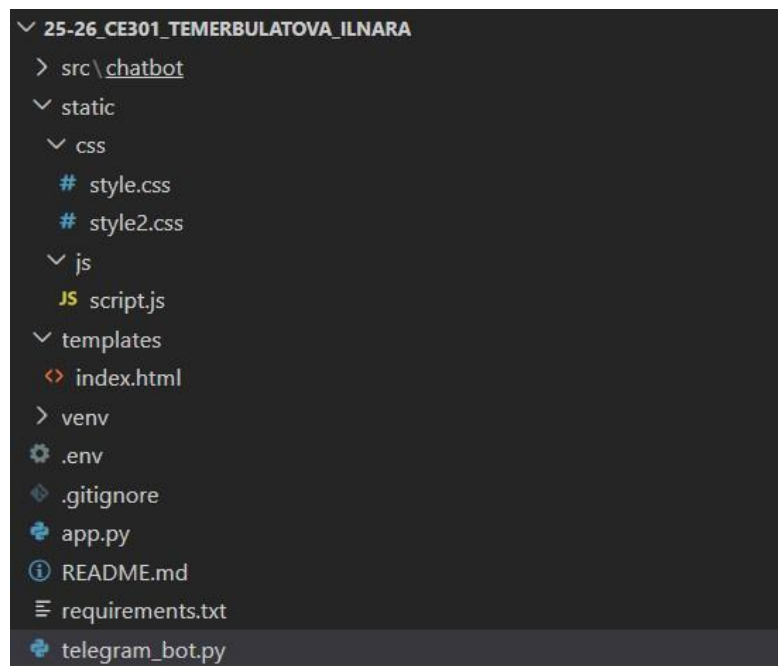


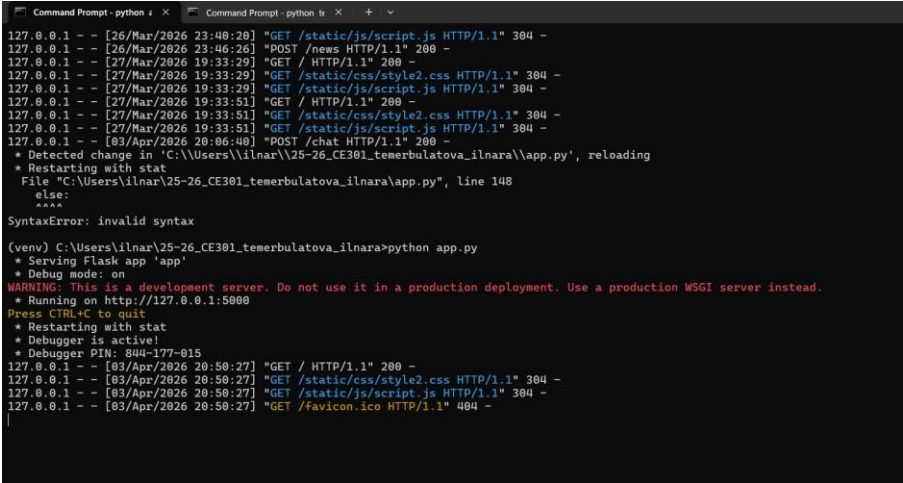
Figure 2. Project Structure in VS Code

Figure 2 shows the project structure within the development environment in this case VS Code. It shows the organization of backend, frontend files that was used during development.

4.2 Backend Implementation

The backend of the project is implemented using Flask web framework. It is the core component of the system, responsible for handling user requests, processing data, and communicating with external services. It connects the web service, Telegram bot, AI model and some APIs as well.

This was implemented in a single file structure during development, with the main logic contained in `app.py`. The server includes several main routes, each one of that is responsible for a different system function. As an example, the `/chat` route processes user messages and generates AI based responses, while `/news` provide access to news services.



```

Command Prompt - python 4 x Command Prompt - python 4 x
127.0.0.1 - - [26/Mar/2026 23:40:20] "GET /static/js/script.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Mar/2026 23:46:26] "POST /news HTTP/1.1" 200 -
127.0.0.1 - - [27/Mar/2026 19:33:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [27/Mar/2026 19:33:29] "GET /static/css/style2.css HTTP/1.1" 304 -
127.0.0.1 - - [27/Mar/2026 19:33:29] "GET /static/js/script.js HTTP/1.1" 304 -
127.0.0.1 - - [27/Mar/2026 19:33:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [27/Mar/2026 19:33:51] "GET /static/css/style2.css HTTP/1.1" 304 -
127.0.0.1 - - [27/Mar/2026 19:33:51] "GET /static/js/script.js HTTP/1.1" 304 -
127.0.0.1 - - [03/Apr/2026 20:06:40] "POST /chat HTTP/1.1" 200 -
* Detected change in 'C:\\Users\\ilnar\\25-26_CE301_temerbulatova_ilnara\\app.py', reloading
* Restarting with stat
File "C:\\Users\\ilnar\\25-26_CE301_temerbulatova_ilnara\\app.py", line 148
    else:
    ^^^^^
SyntaxError: invalid syntax
(venv) C:\\Users\\ilnar\\25-26_CE301_temerbulatova_ilnara>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 804-177-015
127.0.0.1 - - [03/Apr/2026 20:50:27] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [03/Apr/2026 20:50:27] "GET /static/css/style2.css HTTP/1.1" 304 -
127.0.0.1 - - [03/Apr/2026 20:50:27] "GET /static/js/script.js HTTP/1.1" 304 -
127.0.0.1 - - [03/Apr/2026 20:50:27] "GET /favicon.ico HTTP/1.1" 404 -

```

Figure 3. Flask backend running in the terminal

Figure 3 shows the backend server running locally, it confirms that the Flask application is active and handling requests during testing and development. Overall, the backend implementation provides the core processing functionality of the AURA assistant. It connects all the services within a single system, that ensures request to be handled efficiently.

4.3 Frontend Implementation

The frontend of the assistant was developed by using HTML, CSS and JavaScript. Thanks to use of these services, it covers the main environment for user and functions,

such as collecting user input, displaying responses, and managing features in the browser.

The HTML structure covers the main layout of the application, like chat area with input field and sidebar. The CSS structure was used to create a good visual design, improve layout and readability of conversations. The front end follows a single page design, where content is updated dynamically.

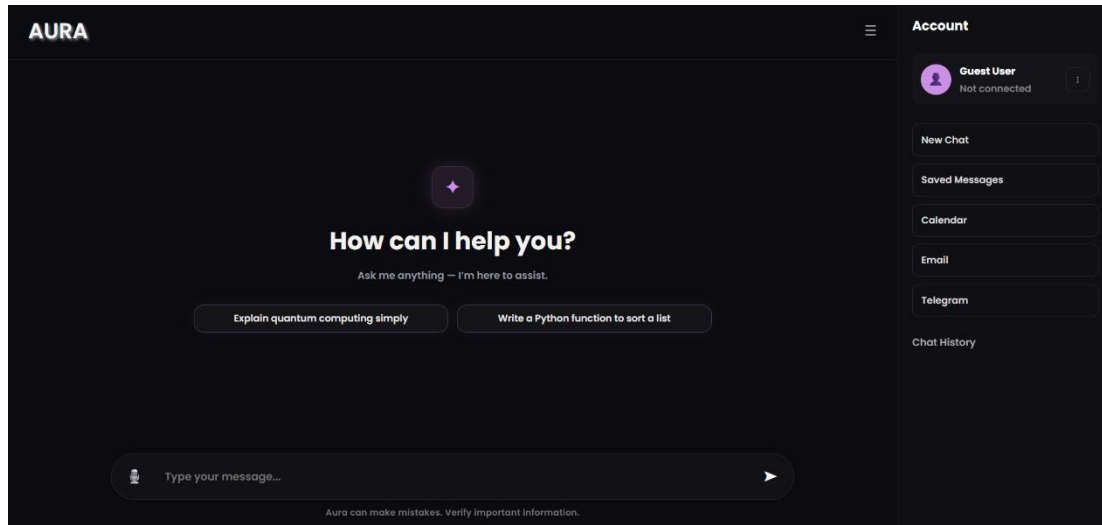


Figure 4. Web Interface of the AURA Assistant

As it can be shown on Figure 4, most of the page covered by the chat area, a text input field with voice input buttons, and a sidebar in order to access the chat history, saved messages, email, calendar and Telegram. Overall, it covers all features to interact in a cleared and well-organized way.

In order to connect and manage the frontend and backend server it is used JavaScript. When a user sends a message, the script takes the text and moves it to the Flask backend.

```
async function sendMessage() {
  const text = input.value.trim();
  if (!text) return;
  hideGreetingSmoothly();
  ensureCurrentChat();
  appendMessage("user", text);
  updateCurrentChatTitle(text);
  input.value = "";
}
```

```

    // If user asks for news
    if (text.toLowerCase().includes("news")) {      const query =
text.replace(/news/gi, "").trim() || "latest";      const news =
await fetchNews(query);      appendMessage("bot", news);
return;
    }      try {      const response = await
fetch("/chat", {      method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ message: text })
    });      const data = await
response.json();
appendMessage("bot", data.reply);
    } catch {      appendMessage("bot", "⚠ Error: Unable to
connect to the server.");
    }
}

```

In the code part, it can be seen the implementation that provides functionality that returns messages without refreshing the page for creating real-time chat experience. Messages are displayed dynamically in the browser using JavaScript, this makes the interaction easier to follow and improves readability.

4.4 Telegram Bot Implementation

The project was extended with a Telegram chatbot in order to provide one more platform to interact with except the web interface. This was implemented with Telegram Bot API and python libraries (python-telegram-bot), that provides a way to manage and organize commands, send and receive messages [6].

At first stage, it was created within telegram bot the Bot Father, which generates the appropriate token. Then for better user experience, additional commands were integrated, such as /start, /help, /about, and /clear (can be seen on Figure 5). The /start command sends greeting message to the user, while /help displays the available commands and their purposes.

```

# /start command async def start(update: Update, context:
ContextTypes.DEFAULT_TYPE):

```

```

await update.message.reply_text(
    "🤖 Hello! I'm your AI Assistant Bot.\n\n"
    "You can chat with me here or use the web app.\n\n"
    "Type anything to start chatting!"
) # /help
command
async def help_command(update: Update, context: ContextTypes.DE-
FAULT_TYPE):
    help_text = (
        "🤖 *Aura AI Assistant*\n\n"
        "Here are the commands you can use:\n\n"
        "/start - Start the bot\n\n"
        "/help - Show this help menu\n\n"
        "/about - About this bot\n\n"
        "/clear - Clear the chat messages\n\n"
        "You can also just type any message and I will respond using
AI."
    )
    await update.message.reply_text(help_text, parse_mode="Markdown")

```

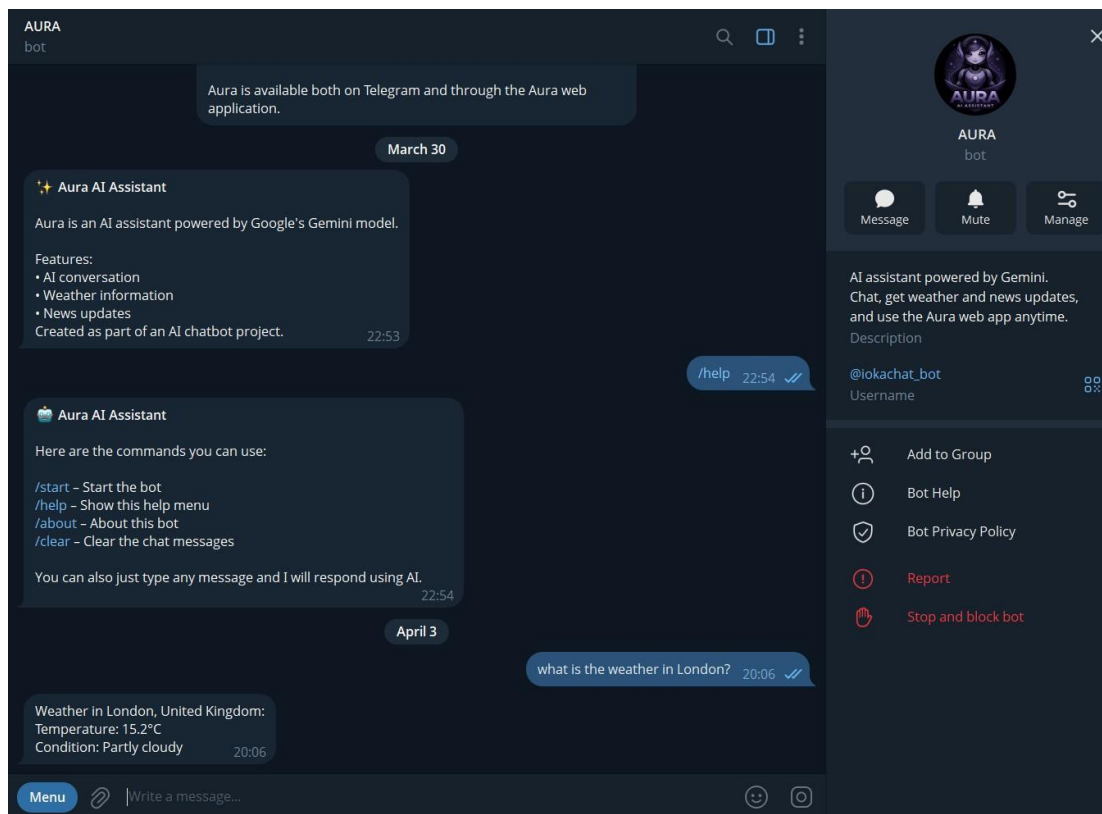


Figure 5. Telegram chatbot interaction with the AURA Assistant.

Figure 5 demonstrates the Bot in use, including commands interaction and normal chatbot responses (in this case /help command and request about weather).

Moreover, beside that commands bot processes normal user messages by sending them to the backend server. The user receives answer from the backend server, which manages the chatbot's logic through Telegram. This structure handles the functionality by using the same backend logic.

4.5 Integration with External APIs

In order to enhance the functionality of the AURA assistant beyond basic conversation, it integrates some external APIs. Thanks to that, users can experience real-time data and AI-generated responses. Each integration was implemented within the Flask backend, where each request is managed by appropriate external services.

The main component in order to receive AI-generated responses, the Google Gemini API was used. When user inputs it requests, the backend server sends it to AI model, after it processes the text and returns back to the user interface. This implementation can be seen provided part of code below:

```
# Regular Gemini chat          response =
client.models.generate_content(
model="gemini-2.5-flash",
contents=user_message
)          bot_reply =
response.text.strip()
return jsonify({"reply":
bot_reply})
```

When it comes to Weather and News APIs integration, that allows users to receive realtime information. The backend server finds weather and news related queries by checking for keywords such as “weather” and “news” in user input. Then it sends to appropriate API, and retrieves structured data for weather and news request as it can be seen on the code below:

```
# Check if the user is asking about weather
if "weather" in user_message.lower():
# Try to extract city after 'in'
words = user_message.split()
location = None
for i, word in enumerate(words):          if
word.lower() == "in" and i + 1 < len(words):
```

```

        location = " ".join(words[i+1:])
    break
    if
location:
        weather_data = get_weather(location)
    if "error" in weather_data:
        bot_reply =
    f"⚠ Could not get weather:
    {weather_data['error']}"
    else:
        bot_reply = (
            f"Weather
in {weather_data['location']}: \n"
            f"Temperature: {weather_data['temp_c']}°C \n"
            f"Condition: {weather_data['condition']}"
        )
    else:
        bot_reply
= "Please specify the city. For example:
'What's the weather in London?'"

query = request.json.get("query", "latest")
news_api_key =
os.getenv("NEWS_API_KEY")
url =
f"https://newsapi.org/v2/everything?q={query}&ap
iKey={news_api_key}"
response =
requests.get(url)
data =
response.json()
if data.get("status") !=
"ok":
    return jsonify({"news": "Sorry, I can't fetch news right
now."})
articles = data.get("articles", [])[0:5]
news_result = ""
for art in
articles:
    news_result += f"◇ **{art['title']}**\n"
news_result += f"{art['description']}\n"
news_result += f"{art['url']}\n\n"
    return jsonify({"news":
news_result})

```

As it seen on the codes part, the retrieved data is formatted into a readable message before returning the output to the user. So, this ensures a well-structured relevant output.

In addition, AURA assistant integrates Google services, such as Gmail and Calendar to provide functional access to these services. This is implemented using OAuth 2.0 authentication, which allows secure access to user data.

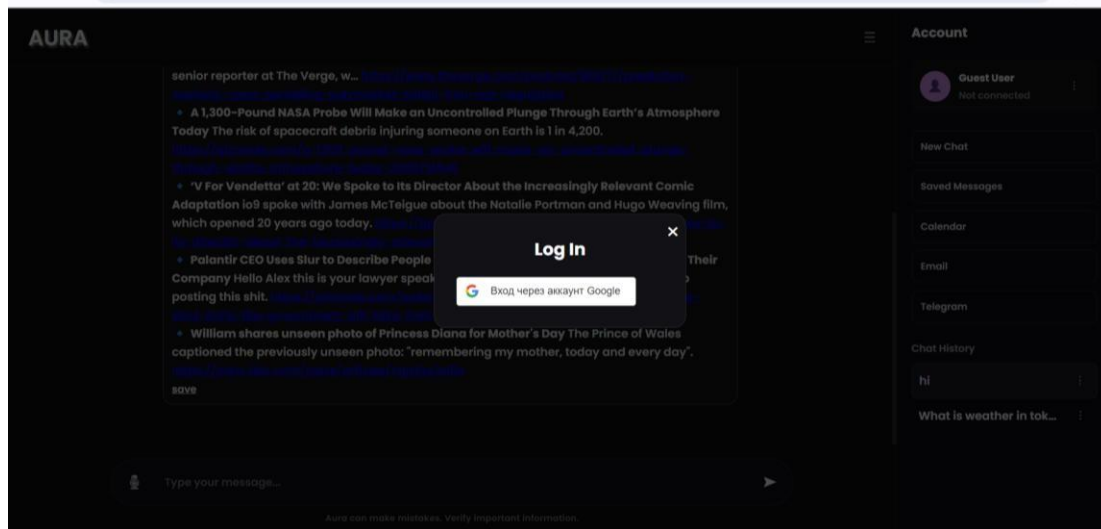


Figure 6. Google login

Figure 6 shows how to log in to the application, Google login authentication is used, to provide safe login.

4.6 Chapter Summary

The project's functionality is significantly improved by the integration of external APIs. Thanks to the combination of AI-generated responses, the system provides a more powerful and useful user experience.

Chapter 5. SYSTEM FEATURES AND FUNCTIONALITY

This chapter covers the main features and functionality of the AURA assistant. Through the integration of conversational AI with various external services, the system is intended to offer an intelligent, engaging, and user-friendly experience. The implemented features show how contemporary AI systems may be used in practical applications to improve productivity, usability, and accessibility [4].

5.1 Conversational AI Capabilities

The core functionality of the AURA assistant is its ability to communicate with users using natural language. Users can submit queries through either the web interface or the Telegram chatbot, and the system generates responses using the Google Gemini large language model [7].

The assistant is capable of:

- Understanding user input in natural language
- Generating context-aware and meaningful responses
- Handling a wide range of queries, including general questions and task-based requests

This functionality enables intuitive human–computer interaction, which is a fundamental characteristic of modern chatbot systems [2].

5.2 Web-Based Interface

The AURA assistant includes a web-based interface developed using HTML, CSS, and JavaScript. The interface provides a chat-oriented environment designed to be simple, intuitive, and user-friendly.

Key features of the web interface include:

- Conversational chat layout
- Sidebar navigation for managing conversations

- Ability to save messages
- Local storage for maintaining chat history

The design focuses on usability principles to ensure efficient interaction and a positive user experience. Research indicates that user-friendly interfaces significantly impact user adoption and satisfaction with conversational AI systems [3].

As shown previously, Figure 4 illustrates the main chat interface, where users can interact with the assistant in real time. The layout supports a conversational flow, allowing messages to be displayed clearly and sequentially. The sidebar provides additional functionality for managing chat sessions and accessing saved messages.

5.3 Telegram Bot Integration

In addition to the web interface, the system provides a Telegram chatbot that allows users to interact with the assistant through a messaging platform. The Telegram Bot API is used to enable communication between the user and the backend server [6].

The Telegram bot:

- Receives user messages
- Sends requests to the backend server
- Returns generated responses to the user

This integration improves accessibility by allowing users to interact with the assistant outside the web environment. Multi-platform accessibility is identified as a key factor in the widespread adoption of conversational AI systems [3].

As shown previously, Figure 5 shows an example of user interaction with the Telegram bot. The chatbot processes user queries and returns responses generated by the AI model, demonstrating the system's multi-platform capability. Additionally, for improved user experience several custom commands were developed:

- **/start** – initiates the chat session
- **/help** – displays available commands and usage information
- **/about** – provides a description of the assistant

- **/clear** – clears the current chat history

5.4 Integration with External Services

To extend the functionality of the conversational assistant except basic dialogue, the system implements integration with several external services through their APIs simultaneously. These integrations allow the chatbot to deliver real time data and interact with the appropriate information [1].

Weather Information Service

The system provides users with the ability to obtain current weather conditions for a specified location. When a user submits a location query, the backend system constructs a request to a weather API service. The system then parses this data and formats it into a structured response that is presented to the user in a natural and readable format. This integration allows the chatbot to deliver accurate, up-to-date environmental information without requiring manual user searches.

News Retrieval Service

The system incorporates a news aggregation feature that enables users to request the latest articles based on a specific topic or keyword. Upon receiving a user request, the system sends a query to a news API, which returns a collection of relevant articles from various sources. The retrieved data includes article titles, publication sources, publication dates, and brief descriptions. The system processes this information and presents a summarized version to the user, allowing for quick consumption of current events without information overload. This feature enhances the chatbot's utility by providing access to dynamic, time-sensitive content.

Google Services Integration

To provide personalized productivity features, the system integrates with Google services, specifically Gmail and Google Calendar. This integration is implemented using OAuth 2.0 authentication protocol, which ensures secure access to user data. The authentication process requires user consent before the system can access any personal information, maintaining security and privacy standards.

Through this integration, the system offers convenient access to their email and calendar applications. Unlike the weather and news integrations which retrieve and display data directly within the chat interface, the Google services integration implements a navigation-based approach.

5.5 Chat Management and History

The system implements comprehensive chat management features that allow users to organize and revisit their conversations effectively. These features are designed to enhance user experience by providing control over interaction history.

Saving Important Messages

The system provides users with the ability to mark and save important messages during a conversation. This feature allows users to highlight specific exchanges that contain valuable information, such as answers to queries, important instructions, or useful data retrieved from external services. Saved messages can be accessed separately, enabling users to quickly reference critical information without scrolling through entire conversation histories.

5.6 Voice Input Functionality

The web interface includes a voice input feature that allows users to interact with the system using speech. This is implemented using browser-based speech recognition technology.

Voice input enhances accessibility and provides an alternative interaction method, particularly for users who prefer hands-free communication.

5.7 Error Handling and System Feedback

To ensure system reliability, basic error handling mechanisms are implemented. When issues occur, such as API failures or connectivity problems, the system provides appropriate feedback to the user.

5.8 Chapter Summary

This chapter presents the system features and functionality of the AURA assistant. The system combines several key components to deliver a comprehensive AI-powered conversational platform. By combining conversational AI, multi-platform accessibility, external service integration, and user management, the system delivers a flexible and practical solution for everyday use.

Overall, this chapter demonstrates how modern AI technologies can be effectively integrated into web-based applications to create intelligent and interactive systems. The AURA assistant serves as an example of a practical application that combines conversational AI with essential supporting features to deliver value to end users [4].

Chapter 6. TESTING AND EVALUATION

This chapter shows the methods for testing the AURA assistant and evaluates how useful and easy it is to use. The purpose of testing is to make sure that all parts of the system work as it should and meets its goals. A combination of functional, integration, usability, and performance testing was carried out.

6.1 Testing Approach

A structured testing method was used to make sure the system worked. Each feature was tested on its own and then as part of the whole system to ensure they worked together correctly. Testing was performed both throughout the development and after implementation to find and fix issues as possible.

The main methods of testing:

- Functional testing
- Integration testing
- User interface testing
- Performance testing

These methods assist make sure that the system is reliable and reduce the errors during operation.

6.2 Functional Testing

Functional testing was performed to be responsible for verification that each feature of the system works as expected. The outputs were compared with the expected outcomes after different user inputs that were given.

Test Case	Input	Expected Output	Actual Output	Status
AI Chat	“Hello”	Relevant response	“Hi there! How can I help you today?”	Pass
Weather Feature	Weather in London?	Relevant “Temperature and condition”	“Weather in London, United Kingdom: Temperature: 10.1°C Condition: Clear”	Pass
News Feature	Latest News	List of relevant news	News articles and links returned	Pass
Login system	Google login	User authenticated	Successful login	Pass
Telegram Bot	“Hello” via bot	AI response	Relevant response returned	Pass

Table 2. Functional Testing Results

Table 2 shows that all main system features function correctly and meet expected requirements.

6.3 Integration Testing

To make sure that different system parts function well together, integration testing was conducted. This method covers communication between external APIs, the backend, and the frontend.

List of integrations that were tested:

- Flask backend connection from a web interface
- Backend communication from Telegram bot

- Gemini API integration
- Backend connection with Gmail, Calendar, Weather and News.

This system successfully handled all data interchange between the components. The same backend logic is used by both the Telegram bot and the web interface, which ensures consistent behaviour across platforms.

6.4 User Interface and Usability Testing

In order to assess the system's responsiveness and usability, user interface testing was completed. This part ensures that the UI is simple and intuitive to use.

Key finding consists of:

- Messages are shown in a conversational layout on the chat interface.
- The side bar is easy to navigate and well-organized
- Saved messages and chat history enhance usability
- Voice input offers one more interaction method

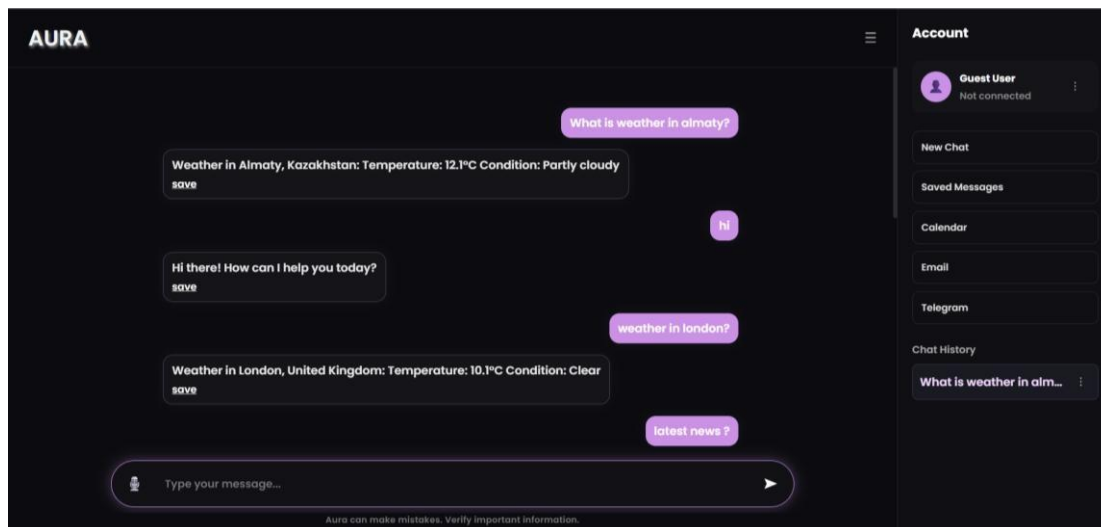


Figure 7. Example of User interaction

As it shown on Figure 7, user interact with the system through the web interface. In addition to facilitating real-time communication, the layout supports clear presentation of user inputs and the system responses

6.5 Performance Testing

Performance Testing was completed to assess the stability and responsiveness of the system. In order to perform this testing, the system was tested under normal usage circumstances by sending several queries and interacting with various features.

AURA assistant demonstrated:

- Quick response times for AI generated responses (usually within few seconds)
- Efficient handling of API request
- Stable performance during ongoing communication

However, a few little delays were identified when retrieving data from external APIs. The overall user experience was not significantly impacted by these delays, though.

6.6 Evaluation and Limitations

The overall performance of the AURA assistant was evaluated based on functionality, usability, and reliability.

The system works well in:

- Generating responses using AI
- Using external APIs to deliver real-time data
- Supporting both platforms interaction (web and Telegram)
- Providing a user-friendly interface

However, several limitations were identified:

- AI-generated responses might occasionally be inaccurate (warning line of text were implemented under the input field)
- The system relies on outside APIs, which might fail or become unavailable
- Performance may be affected by internet connection

Despite these drawbacks, the system fulfils the project's objectives and provides reliable and functional AI assistant.

6.7 Chapter Summary

This chapter covers the testing and evaluation results of this project. All main features were successfully integrated and tested, thanks to that the system provides a stable and user-friendly experience. Overall, this evaluation shows that the system is a successful implementation of conversational assistant that can combine different technologies to provide useful features.

Chapter 7. PROJECT PLANNING AND RISK MANAGEMENT

One of the main parts of the project is to create a plan and follow a structure, allowing to adhere to the plan throughout the project lifecycle. A clear development strategy was established at the beginning of the project to ensure that all components have been completed within the given timeline.

7.1 The Planning of the Project

On the first stages, the structured plan was created in order to organise project into manageable phases. This strategy helped to track the possibility of overlooking important elements during development and helped to sustain progress. The project was divided into several stages:

1. Made research and analysis about the topic and requirements
2. System design and architecture planning
3. Building the backend server
4. Design and develop the web interface
5. Connecting the AI model and APIs
6. Setting up the Telegram bot
7. Testing and debugging

Rather than trying to build everything at once, I decided to work through one stage at a time: complete one step, test it, fix any issues and then move on. This approach helped to catch the problems early rather than discover the issues at the end when everything is already connected.

7.2 Tuleap Project Activity

Throughout the development process, Tuleap was used as a project management tool. It lets me set up task lists, define milestones, and get a visual overview of what was done and what still needed doing.

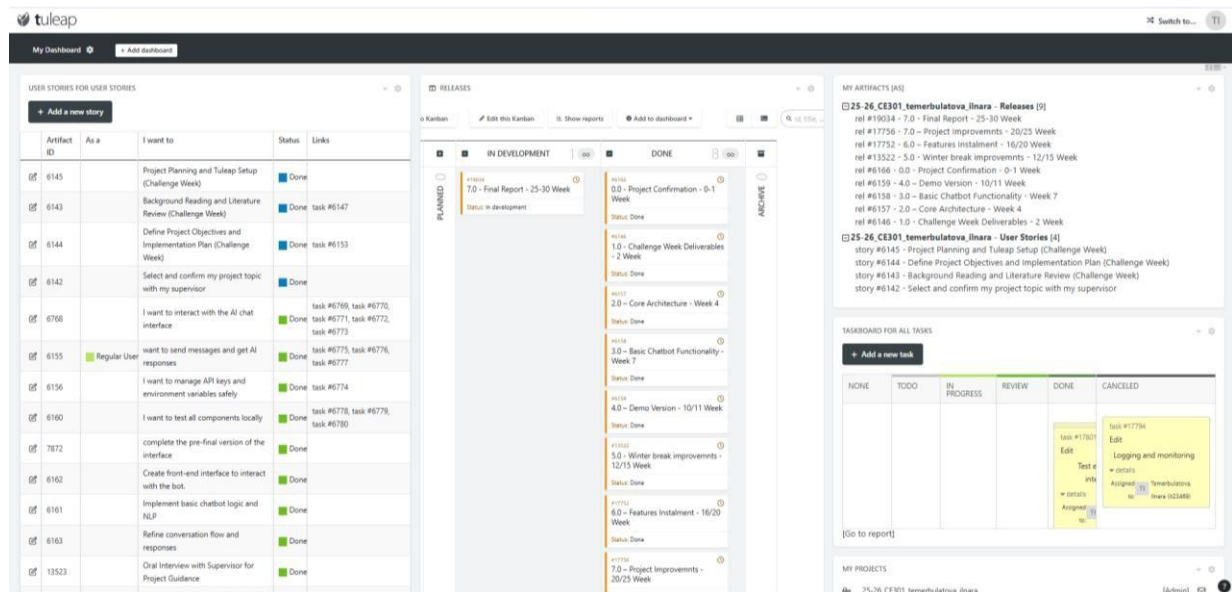


Figure 8. Tuleap Dashboard

As shown in Figure 8, Tuleap enabled:

- Creation of task lists
- Definition of milestones for main deadlines
- Visual tracking board of progress

Using Tuleap helped me to see a clear picture of the progress that have been done and what still needed doing and provided clear overview of the project timeline. Moreover, it supported communication with the project supervisor, who had access to the platform for monitoring development.

7.3 Version Control with GitLab

Version control was handled through GitLab (Figures 9 and 10). Committing changes regularly meant I always had a working version to fall back on, which was particularly useful when an integration did not go as planned.

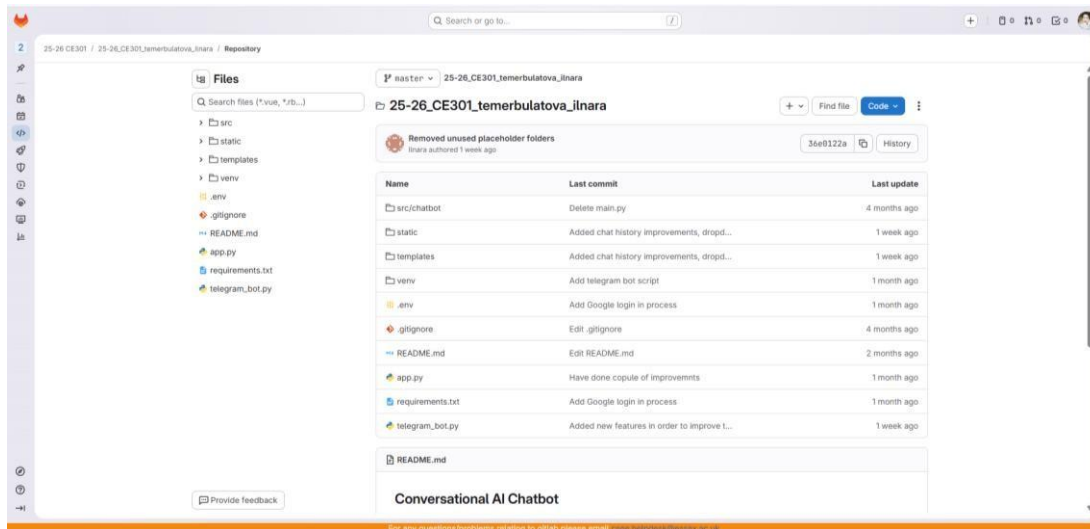


Figure 9. GitLab Structure

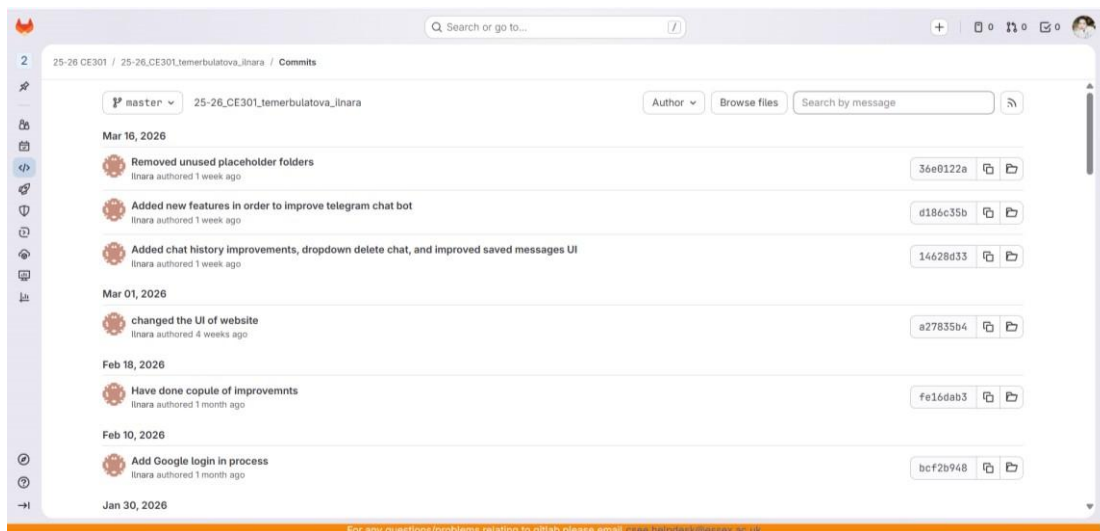


Figure 10. GitLab Commits

GitLab was used for:

- Store and manage the project code
- Track changes through commits (Figure 9)

- Have a back of project files

The use of GitLab ensured that the code changes would be documented and committed, it reduced the risk of data loss.

7.4 Time Management

Time management was a crucial aspect of the project. To guarantee that every stage of development was finished within the allotted time, a defined timeline was adhered to.

I tried to give each part of the project a rough deadline and checked my progress against it. Tuleap helped with this because I could see if I was falling behind on anything. There were a couple of times I had to adjust my plans when something took longer than expected, but because I was tracking everything, I could shift things around without panicking.

7.5 Risk Identification

During the development of the project, several potential risks were identified. These risks could affect system performance, functionality or project completion.

The main risks included:

- Failure of integrating external APIs
- Delays because of integration complexity
- Errors in AI generated answers
- Risks related to user authentication
- Performance issues

In order to avoid and minimize the impact of the risks, to handle and avoid these risks it was chosen to use safe and proven methods such as error handling, testing etc.

7.6 Chapter Summary

Looking back, using Tuleap and GitLab together made a real difference. The Aura assistant was successfully developed within the project restrictions because of the combination of careful planning, continuous monitoring, and risk management.

Chapter 8. PROJECT CONTEXT

This chapter covers the broader background of the AURA assistant, with particular attention to the legal, ethical, health and safety aspects. These elements are crucial for creating AI conversational systems since they have an impact on how people interact with the system.

8.1 Legal Considerations

Since the AURA assistant engages with users and integrates others external services like Google API's, there are some important legal aspects.

Data protection is one of the main issues. Despite being only a prototype, this project adheres to fundamental concepts including use of secure authentication methods like Google login and keeping sensitive user data. This helps reduce risks related to data leakage.

Moreover, it is crucial to ensure users, that their data is handled safely throughout conversations, namely never being shares without consent. This project uses a small amount of data, and most of the process is done by other services that already adhere to their own security guidelines.

Additionally, system must follow the terms and conditions of external APIs. Such as, Google Gemini, Gmail and Calendar APIs.

8.2 Ethical Considerations

Also, dealing with AI systems it is very important to mention ethical considerations, when systems communicate directly with people.

One of the main aspects is the accuracy of replies produced by AI. This assistant is not an exception, where it uses a LLM that can sometimes produce inaccurate or unclear answers. This implies that users should be aware of the system's limits and refrain from using it for important choices.

Furthermore, another important issue is having transparent aspect. The AURA project makes it obvious that it is an AI-based system, so users are aware of communicating with a human.

8.3 Health and Safety Considerations

When it comes to Health and Safety considerations, there are no direct physical dangers as the project is software-based system. However, there are still some relevant health and safety issues that may occur.

Form a development perspective, long hours of working on a computer can lead to eye strain and exhaustion. While developing this project, some fundamentals procedures like taking breaks and keeping proper posture were followed.

8.4 Chapter Summary

The main ideas of legal, ethical and safety considerations have been covered in this chapter. Even though the system is a prototype, it adheres to basic design principles.

Overall, these factors show that creating AI system involves more than technical implementation but also ensures that the system is secure and suitable for users.

Chapter 9. RESULTS AND DISCUSSION

This chapter describes the outputs obtained from the development and testing of the AURA assistant and provides a discussion of its overall performance. The main goal is to evaluate how effectively the system meets its objectives and to analyze its strengths and limitations.

9.1 Overview of Project Objectives

The main idea of project objectives was to develop AI-powered conversational assistant, that works across two platforms and can pull in real-time data from external services.

The system successfully follows its main objectives, based on the features and the results presented in Chapter 5 and 6 simultaneously. This project supports the ability to process user input, generate AI-based responses, and retrieve real-time information such as weather and news. Additionally, the implementation of two platforms (web and Telegram) enables access to effective interaction, and this confirms that project is suitable for real-world applications.

9.2 System Effectiveness and Performance

The AURA assistant demonstrates a high level of effectiveness at doing what it was designed to do. The solution works well because it can handle a wide range of user questions by combining AI-generated answers with data from external APIs.

A client-server design makes the system more flexible and easier to manage. The technology makes sure that the web interface and Telegram platform behave the same way by processing within the backend. This design also allows for scalability because it would be easy to add more interfaces with only a few changes.

In terms of performance, the system operates reliably under normal usage conditions. Response times differ based on the kind of request, as seen during testing (Chapter 6). Responses with AI generation take longer due to processing complexity, while API

based replies like weather and news are provided faster. It is important to note that system performance is under the control of outside systems, including network connectivity and API availability.

9.3 Strengths of the System

The outcomes demonstrate a few of the AURA's assistant's main advantages:

- AI's effective integration with several external services
- Support for both web-based and telegram-based interaction
- An intuitive and user-friendly UI
- System architecture that is scalable and modular

These strength highlights that the project achieves a balance between accessibility and technical functionality, which is crucial for nowadays AI applications.

9.4 Limitations and Challenges

On the other hand, the system has a number of limitations that were discovered during development.

One of the main limitations is the use of AI-generated replies, which might occasionally produce insufficient information. Also, reliance on outside APIs another disadvantage. If some of these services are unavailable or experience delays, the systems functionality may be impacted. Additionally, the system's scalability and possibilities for real-world implementation are limited due to it's still a locally installed prototype. To handle more users, more sophisticated infrastructure would be needed.

9.5 Chapter Summary

In summary, the results show that the AURA assistant is a successful implementation of a conversational AI system within its intended objectives and also demonstrates good quality performance across its core features. This project provides a practical example

of how system effectively combines different technologies that can be integrated into modern applications and deliver a functional and interactive user-experience.

Chapter 10. CONCLUSION

The main idea of this project was to create and build a conversational AI chatbot that works on both platforms and delivers useful, real-time features to users. The AURA assistant has been successfully put into use and meets the main goals that were set at the start of the project.

The system fulfils its core requirements: it processes natural language input, uses an AI model to generate the responses, and retrieve relevant real-time information. Moreover, the system can be used through different ways, which makes it more flexible and useful. As we know, it works with external services like Google and has web and Telegram interfaces.

One of the biggest successes during the development process was putting together different technologies into one working system. The Flask backend architecture proved effective in coordinating communication between the AI model, external APIs, and two separate user interfaces. Testing confirmed that all core features function as expected, and the structured development approach using Tuleap for planning and GitLab for version control helped identify and resolve issues at each stage rather than at the end.

However, there are some challenges that have been faced during the development. The system relies on third-party APIs, so it might not always work well if those services are down. Also, AI-generated answers can also be inaccurate at times, especially for more complicated questions. As a locally hosted prototype, it is not yet ready for large-scale deployment.

In terms of future work, there are a number of ways that the system could be made better. One of the steps would be deploying the system to a cloud platform to improve accessibility and scalability and improving conversation memory to make interactions feel more personalized over time.

Overall, the AURA assistant represents a complete and working implementation of a conversational AI system built within the scope of a Bachelor's project. The AURA assistant is a real-life example of a conversational system that uses AI, we technologies,

and external services. The project meets its goals and lays a strong foundation for future work, even though there some areas that could be better.

REFERENCE LIST

- [1] Mohammed, M. and Aref, M.M. (2022). *Chatbot System Architecture*. [online] arXiv.org. Available at: https://arxiv.org/abs/2201.06348?utm_source=chatgpt.com [Accessed 30 Mar. 2026].
- [2] Zhou, L., Gao, J., Li, D. and Shum, H.-Y. (2018). *The Design and Implementation of XiaoIce, an Empathetic Social Chatbot*. [online] arXiv.org. Available at: https://arxiv.org/abs/1812.08989?utm_source=chatgpt.com.
- [3] Wang, X., Wang, X., Park, S. and Yao, Y. (2025). Users' Mental Models of Generative AI Chatbot Ecosystems. *arXiv (Cornell University)*. [online] doi:<https://doi.org/10.1145/3708359.3712125>.
- [4] Aldhafeeri, L., Aljumah, F., Thabyan, F., Alabbad, M., AlShahrani, S., Alanazi, F. and AlNafjan, A. (2025). Generative AI Chatbots Across Domains: A Systematic Review. *Applied Sciences*, [online] 15(20), p.11220. doi:<https://doi.org/10.3390/app152011220>.
- [5] Madi, A. and Al-Ajlan, A. (2023). Chatbot Development using APIs and Integration into the MOOC. *International Journal of Interactive Mobile Technologies*, Vol. 5, No. 1. doi:<https://doi.org/10.36427/cejntrep.5.1.5041>.
- [6] core.telegram.org. (n.d.). *Telegram Bot API*. [online] Available at: <https://core.telegram.org/bots/api>.
- [7] Google AI for Developers. (n.d.). *Build with the Gemini API*. [online] Available at: <https://ai.google.dev/>.
- [8] DALE, R. (2016). The return of the chatbots. *Natural Language Engineering*, 22(5), pp.811–817. doi:<https://doi.org/10.1017/s1351324916000243>.
- [9] Kovan Mzwri and Márta Turcsányi-Szabó (2023). Chatbot Development using APIs and Integration into the MOOC. 5(1), pp.18–30. doi:<https://doi.org/10.36427/cejntrep.5.1.5041>.